

```
best_params = study.best_params
best_value = study.best_value
```

Common Pitfalls to Avoid:

Pitfall	Solution
Too few trials	Use at least 30-50 trials
Too narrow search space	Expand ranges initially
Forgetting .to(device)	Move model and data to GPU
Not flattening input	Use x.view(x.size(0), -1)
Wrong direction	Maximize accuracy, minimize loss

Practice Exercise

Try to tune these additional hyperparameters:

1. **Activation functions** (ReLU, LeakyReLU, ELU)
2. **Number of CNN layers** (if using CNN instead of ANN)
3. **Kernel sizes** (for CNN)
4. **Learning rate schedulers**

Final Notes

- This same approach works for **CNN, RNN, Transformers** - any PyTorch model
- Optuna is industry-standard (used at Google, Microsoft, etc.)
- Always combine hyperparameter tuning with experiment tracking
- Start with small number of trials, then increase once code works

Remember: The goal is not just to get the highest accuracy, but to understand how different hyperparameters affect your model's performance on your specific dataset!

Building a CNN using PyTorch - Complete Notes

Video Source: CampusX Video 11

Topic 1: Recap of Previous Work on Fashion-MNIST

In-Depth Explanation

Before building a CNN, let's understand what was done earlier with Artificial Neural Networks (ANN) on the Fashion-MNIST dataset:

Version 1 (CPU - Subset Data):

- Used only a subset of the complete dataset because training was very slow on CPU
- Accuracy achieved: ~83%

Version 2 (GPU - Full Dataset):

- Switched to GPU for faster training
- Used the complete dataset
- Accuracy improved to: 88.9%
- **Problem Identified:** Training accuracy was ~98% while test accuracy was 88.9% → **Overfitting**

Version 3 (Optimization - Batch Norm & Dropout):

- Added Batch Normalization and Dropout layers
- Training accuracy reduced to ~93% (less overfitting)
- Test accuracy remained at 88.9%

Version 4 (Hyperparameter Tuning with Optuna):

- Used Optuna library for hyperparameter optimization
- Accuracy reached: ~90%

Key Limitation of ANNs: ANNs don't perform very well on image-based data. This is why we need CNNs!

Code (Recap - Not executed, just understanding)

```
# This is just to understand the progression
# Version 2 ANN Model (simplified concept)
class ANN(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784, 128)
        self.fc2 = nn.Linear(128, 64)
        self.fc3 = nn.Linear(64, 10)

    def forward(self, x):
        x = x.view(-1, 784) # Flatten the image
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

Output (Conceptual)

Version 2: Test Accuracy = 88.9%, Train Accuracy = 98% (Overfitting)
Version 3: Test Accuracy = 88.9%, Train Accuracy = 93% (Less overfitting)
Version 4: Test Accuracy = ~90%, Train Accuracy = ~95%

Topic 2: What are CNNs? (Convolutional Neural Networks)

In-Depth Explanation

Definition: CNNs are a type of neural network specifically designed to work well with grid-based data like images and videos.

Why are CNNs good for images?

- Images can be represented as a grid of pixels (numbers)
- Each pixel has a numerical value
- CNNs can identify spatial patterns in these grids

CNN Architecture - Two Main Stages:

Stage 1: FEATURE EXTRACTION

- └ Convolution Layer (extracts low-level features like edges)
- └ Pooling Layer (reduces size of the representation)

Stage 2: CLASSIFICATION

- └ Fully Connected Neural Network (same as ANN)

Detailed Breakdown of Feature Extraction:

1. **Convolution Operation:**
 - Small filters (e.g., 3×3) slide over the image
 - Each filter detects specific patterns (edges, curves, etc.)
 - Multiple filters can be used in one layer (32, 64, 128, etc.)
 - Produces feature maps
2. **Pooling Operation (usually MaxPooling):**
 - Reduces the spatial size of the feature maps
 - Common size: 2×2 with stride 2 (reduces size by half)
 - Helps make the model more robust to small translations

Complete CNN Architecture Flow:

Input Image → [Conv → Activation → Pool] repeated N times → Flatten → [Fully Connected Layers] → Output

Code (Conceptual Architecture)

```
# Pseudo-code showing CNN structure
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        # Feature Extraction Stage
        self.features = nn.Sequential(
            nn.Conv2d(in_channels, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2)
        )

        # Classification Stage
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64*7*7, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 10)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x
```

Output (No output - conceptual understanding)

Topic 3: Our CNN Architecture for Fashion-MNIST

In-Depth Explanation

Input Specifications:

- Image size: 28×28 pixels
- Channels: 1 (grayscale/black & white)
- Input shape: (1, 28, 28)

Architecture Design:

Layer	Type	Details	Output Shape
Input	Image	1 channel, 28×28	(1, 28, 28)
Conv1	Conv2d	32 filters, 3×3,	(32, 28, 28)

Layer	Type	Details	Output Shape
		padding=1	
Activation	ReLU	-	(32, 28, 28)
Pool1	MaxPool2d	2×2, stride=2	(32, 14, 14)
Conv2	Conv2d	64 filters, 3×3, padding=1	(64, 14, 14)
Activation	ReLU	-	(64, 14, 14)
Pool2	MaxPool2d	2×2, stride=2	(64, 7, 7)
Flatten	-	Reshape to 1D	(64×7×7 = 3136)
FC1	Linear	3136 → 128	(128)
Dropout	Dropout	p=0.5 (optional)	(128)
FC2	Linear	128 → 64	(64)
Dropout	Dropout	p=0.5 (optional)	(64)
FC3	Linear	64 → 10	(10)

Why these numbers?

- 32 and 64 filters: Standard practice, doubling each time
- 3×3 kernels: Common choice for capturing local patterns
- Padding=1: Maintains spatial dimensions (28×28 stays 28×28)
- 2×2 pooling with stride 2: Reduces size by half each time

Code (Architecture Definition)

```
import torch
import torch.nn as nn

class CNN(nn.Module):
    def __init__(self, input_channels=1, learning_rate=0.01):
        super().__init__()

        # FEATURE EXTRACTION STAGE
        self.features = nn.Sequential(
            # First Conv-Pool Pair
            nn.Conv2d(input_channels, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.BatchNorm2d(32), # Optional: improves training
            nn.MaxPool2d(2, 2), # 28x28 -> 14x14

            # Second Conv-Pool Pair
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.BatchNorm2d(64),
            nn.MaxPool2d(2, 2) # 14x14 -> 7x7
        )
```

```

# CLASSIFICATION STAGE
self.classifier = nn.Sequential(
    nn.Flatten(),
    nn.Linear(64 * 7 * 7, 128), # 3136 -> 128
    nn.ReLU(),
    nn.Dropout(0.5), # Prevents overfitting

    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Dropout(0.5),

    nn.Linear(64, 10) # 10 output classes
)

def forward(self, x):
    x = self.features(x)
    x = self.classifier(x)
    return x

```

Output (No output - architecture definition only)

Topic 4: Data Preparation for CNN

In-Depth Explanation

Key Change from ANN to CNN:

In ANN, images were flattened to 1D vectors (784 pixels). In CNN, we need to preserve the 2D spatial structure of images.

What needs to change?

1. Input shape: From (784) to (1, 28, 28)
2. Batch dimension: Use -1 as placeholder for batch size
3. Channel dimension: 1 for grayscale, 3 for RGB

Why reshape?

- CNNs expect input in format: (batch_size, channels, height, width)
- Fashion-MNIST images are 28×28 grayscale → 1 channel
- Shape becomes (batch_size, 1, 28, 28)

Code (Data Preparation)

```

import torch
from torch.utils.data import DataLoader, TensorDataset

```

```

# Assuming X_train, X_test, y_train, y_test are loaded

# Reshape data for CNN - THIS IS THE KEY CHANGE
# Original shape: (60000, 784) for training
# New shape: (60000, 1, 28, 28)
X_train = X_train.reshape(-1, 1, 28, 28)
X_test = X_test.reshape(-1, 1, 28, 28)

# Convert to PyTorch tensors
X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.long)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test, dtype=torch.long)

# Create datasets
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
test_dataset = TensorDataset(X_test_tensor, y_test_tensor)

# Create data loaders
batch_size = 32
train_loader = DataLoader(train_dataset, batch_size=batch_size,
                           shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size,
                          shuffle=False)

print(f"Train data shape: {X_train_tensor.shape}") # (60000, 1, 28,
28)
print(f"Test data shape: {X_test_tensor.shape}")   # (10000, 1, 28,
28)

```

Output

```

Train data shape: torch.Size([60000, 1, 28, 28])
Test data shape: torch.Size([10000, 1, 28, 28])

```

Topic 5: Training the CNN Model

In-Depth Explanation

Training Process: The training loop remains almost identical to ANN training. Only the model architecture changes.

Components of Training:

1. **Loss Function:** CrossEntropyLoss (for multi-class classification)
2. **Optimizer:** Adam or SGD (learning rate = 0.01)
3. **Number of Epochs:** 100 (as used in the video)

4. **Device:** GPU (cuda) if available, else CPU

Why does CNN train faster?

- Parameter sharing in convolution layers
- Fewer parameters than a fully connected network for the same input size

Code (Complete Training Pipeline)

```
import torch.optim as optim

# Set device
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# Create model instance
model = CNN(input_channels=1).to(device)

# Loss function and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

# Training parameters
epochs = 100

# Training loop
train_losses = []
train_accuracies = []

for epoch in range(epochs):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)

        # Forward pass
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)

        # Backward pass
        loss.backward()
        optimizer.step()

        # Statistics
        running_loss += loss.item()
        _, predicted = output.max(1)
        total += target.size(0)
```

```

        correct += predicted.eq(target).sum().item()

    epoch_loss = running_loss / len(train_loader)
    epoch_acc = 100. * correct / total
    train_losses.append(epoch_loss)
    train_accuracies.append(epoch_acc)

    if (epoch + 1) % 10 == 0:
        print(f'Epoch [{epoch+1}/{epochs}], Loss: {epoch_loss:.4f},
Accuracy: {epoch_acc:.2f}%')
    print("Training completed!")

```

Output (Approximate - based on video)

```

Using device: cuda
Epoch [10/100], Loss: 0.3124, Accuracy: 88.45%
Epoch [20/100], Loss: 0.1856, Accuracy: 94.23%
Epoch [30/100], Loss: 0.1345, Accuracy: 96.78%
Epoch [40/100], Loss: 0.1045, Accuracy: 97.89%
Epoch [50/100], Loss: 0.0823, Accuracy: 98.56%
...
Epoch [100/100], Loss: 0.0234, Accuracy: 99.92%
Training completed!

```

Topic 6: Evaluating the CNN Model

In-Depth Explanation

Evaluation Process: After training, we evaluate the model on:

1. **Test Data** (unseen data) - to check generalization
2. **Training Data** - to check for overfitting

Results from the Video:

- **Test Accuracy:** 92.6%
- **Training Accuracy:** 99.9%

Analysis:

- Test accuracy improved from 90% (ANN) to 92.6% (basic CNN)
- This is a significant improvement with just a basic CNN architecture
- Some overfitting still exists (99.9% train vs 92.6% test)
- Overfitting is expected because Fashion-MNIST has limited variation

Why is CNN better than ANN?

- CNNs preserve spatial relationships
- Parameter sharing reduces overfitting
- Better at extracting relevant features from images

Code (Evaluation)

```
# Evaluation function
def evaluate(model, data_loader, dataset_name="Test"):
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for data, target in data_loader:
            data, target = data.to(device), target.to(device)
            output = model(data)
            _, predicted = output.max(1)
            total += target.size(0)
            correct += predicted.eq(target).sum().item()

    accuracy = 100. * correct / total
    print(f'{dataset_name} Accuracy: {accuracy:.2f}%')
    return accuracy

# Evaluate on test data
test_accuracy = evaluate(model, test_loader, "Test")

# Evaluate on training data (to check overfitting)
train_accuracy = evaluate(model, train_loader, "Training")

print(f"\nSummary:")
print(f"Training Accuracy: {train_accuracy:.2f}%")
print(f"Test Accuracy: {test_accuracy:.2f}%")
print(f"Overfitting Gap: {train_accuracy - test_accuracy:.2f}%")
```

Output

```
Test Accuracy: 92.60%
Training Accuracy: 99.90%
```

```
Summary:
Training Accuracy: 99.90%
Test Accuracy: 92.60%
Overfitting Gap: 7.30%
```

Topic 7: Complete Code - Putting It All Together

In-Depth Explanation

Here's the complete, ready-to-run code that combines everything we learned. This code will:

1. Load and preprocess Fashion-MNIST data
2. Reshape it for CNN input
3. Build the CNN architecture
4. Train the model
5. Evaluate and display results

Code (Complete Implementation)

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
import numpy as np
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split

# =====
# 1. LOAD AND PREPARE DATA
# =====
print("Loading Fashion-MNIST data...")
fashion_mnist = fetch_openml('Fashion-MNIST', version=1,
                             parser='auto')
X, y = fashion_mnist.data.values, fashion_mnist.target.values

# Convert to numpy arrays
X = np.array(X)
y = np.array(y, dtype=np.int64)

# Normalize pixel values (0-255 to 0-1)
X = X / 255.0

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=10000, random_state=42
)

print(f"Training samples: {X_train.shape[0]}")
print(f"Test samples: {X_test.shape[0]}")

# =====
# 2. RESHAPE FOR CNN (KEY STEP)
# =====
# Reshape from (n_samples, 784) to (n_samples, 1, 28, 28)
X_train = X_train.reshape(-1, 1, 28, 28)
```

```

X_test = X_test.reshape(-1, 1, 28, 28)

print(f"Reshaped X_train: {X_train.shape}") # (60000, 1, 28, 28)
print(f"Reshaped X_test: {X_test.shape}") # (10000, 1, 28, 28)

# Convert to PyTorch tensors
X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.long)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test, dtype=torch.long)

# Create datasets and dataloaders
batch_size = 32
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
test_dataset = TensorDataset(X_test_tensor, y_test_tensor)

train_loader = DataLoader(train_dataset, batch_size=batch_size,
                           shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size,
                          shuffle=False)

# =====
# 3. DEFINE CNN ARCHITECTURE
# =====
class CNN(nn.Module):
    def __init__(self, input_channels=1):
        super().__init__()

        # Feature Extraction Stage
        self.features = nn.Sequential(
            # First convolution block
            nn.Conv2d(input_channels, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.BatchNorm2d(32),
            nn.MaxPool2d(2, 2), # 28x28 -> 14x14

            # Second convolution block
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.BatchNorm2d(64),
            nn.MaxPool2d(2, 2) # 14x14 -> 7x7
        )

        # Classification Stage
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 7 * 7, 128), # 3136 -> 128
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(128, 64),

```

```

        nn.ReLU(),
        nn.Dropout(0.5),
        nn.Linear(64, 10)  # 10 classes
    )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x

# =====
# 4. SETUP TRAINING
# =====
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"\nUsing device: {device}")

model = CNN(input_channels=1).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

epochs = 100
print(f"Training for {epochs} epochs...")

# =====
# 5. TRAINING LOOP
# =====
for epoch in range(epochs):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for data, target in train_loader:
        data, target = data.to(device), target.to(device)

        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        _, predicted = output.max(1)
        total += target.size(0)
        correct += predicted.eq(target).sum().item()

    if (epoch + 1) % 10 == 0:
        epoch_loss = running_loss / len(train_loader)
        epoch_acc = 100. * correct / total
        print(f'Epoch [{epoch+1}/{epochs}], Loss: {epoch_loss:.4f},

```

```

Acc: {epoch_acc:.2f}%')

print("Training completed!")

# =====
# 6. EVALUATION
# =====
def evaluate(model, data_loader, name):
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for data, target in data_loader:
            data, target = data.to(device), target.to(device)
            output = model(data)
            _, predicted = output.max(1)
            total += target.size(0)
            correct += predicted.eq(target).sum().item()

    accuracy = 100. * correct / total
    print(f'{name} Accuracy: {accuracy:.2f}%')
    return accuracy

print("\n" + "="*40)
print("EVALUATION RESULTS")
print("="*40)

test_acc = evaluate(model, test_loader, "Test")
train_acc = evaluate(model, train_loader, "Training")

print(f"\nSummary:")
print(f"✓ Test Accuracy: {test_acc:.2f}%")
print(f"✓ Training Accuracy: {train_acc:.2f}%")
print(f"✓ Overfitting Gap: {train_acc - test_acc:.2f}%")

```

Output (Expected)

```

Loading Fashion-MNIST data...
Training samples: 60000
Test samples: 10000
Reshaped X_train: (60000, 1, 28, 28)
Reshaped X_test: (10000, 1, 28, 28)

Using device: cuda
Training for 100 epochs...
Epoch [10/100], Loss: 0.3124, Acc: 88.45%
Epoch [20/100], Loss: 0.1856, Acc: 94.23%
Epoch [30/100], Loss: 0.1345, Acc: 96.78%
Epoch [40/100], Loss: 0.1045, Acc: 97.89%

```

```
Epoch [50/100], Loss: 0.0823, Acc: 98.56%
Epoch [60/100], Loss: 0.0678, Acc: 98.92%
Epoch [70/100], Loss: 0.0543, Acc: 99.21%
Epoch [80/100], Loss: 0.0432, Acc: 99.45%
Epoch [90/100], Loss: 0.0345, Acc: 99.67%
Epoch [100/100], Loss: 0.0234, Acc: 99.92%
Training completed!
```

```
=====
EVALUATION RESULTS
=====
```

```
Test Accuracy: 92.60%
Training Accuracy: 99.90%
```

```
Summary:
✓ Test Accuracy: 92.60%
✓ Training Accuracy: 99.90%
✓ Overfitting Gap: 7.30%
```

Summary Table: ANN vs CNN Results

Model	Test Accuracy	Training Accuracy	Overfitting Gap
ANN (Version 2)	88.9%	98%	9.1%
ANN (Version 3)	88.9%	93%	4.1%
ANN (Version 4 - Optimized)	~90%	~95%	~5%
CNN (Basic)	92.6%	99.9%	7.3%

Key Takeaway: Even a basic CNN outperforms an optimized ANN on image data!

Key Vocabulary for Your Notes

Term	Meaning
CNN	Convolutional Neural Network - specialized for grid data
Convolution	Operation where filters slide over input to extract features
Filter/Kernel	Small matrix that detects specific patterns
Pooling	Reduces spatial dimensions (usually MaxPooling)
Feature Extraction	First stage of CNN that identifies patterns
Flatten	Converts 2D/3D tensor to 1D for fully connected layers
Padding	Adding zeros around the input to preserve dimensions
Stride	How many pixels the filter moves each step

Term	Meaning
Channels	Color dimensions (1 for grayscale, 3 for RGB)
Overfitting	Model learns training data too well, fails on new data
Dropout	Randomly turns off neurons to prevent overfitting
BatchNorm	Normalizes layer outputs for stable training

Next Steps (Preview of Video 12 & 13)

1. **Video 12:** Optimize CNN architecture further + Hyperparameter tuning with Optuna (Expected accuracy: 93-94%)
2. **Video 13:** Transfer Learning using pre-trained models like ResNet or VGGNet (Even better accuracy!)

Important Notes for Beginners

1. **Prerequisites:** Understand CNN theory before implementing. Watch the recommended 6 videos on CNN fundamentals.
2. **GPU Recommended:** Training CNNs on CPU is very slow. Use Google Colab with GPU runtime.
3. **Shape Matters:** Always remember CNN input shape: (batch_size, channels, height, width)
4. **Overfitting is Normal:** Some overfitting is expected, especially with smaller datasets.
5. **Basic CNN is Powerful:** Even a simple CNN can outperform a complex ANN on image tasks.

End of Notes - Happy Learning!

Here are the detailed notes from the video "**Transfer Learning using PyTorch | Video 12 | CampusX**". You can copy these directly into your notebook.
